MARS: A Memory-Augmented Agentic Terminal Multiplexer that Learns From Your Commands

Anjishnu Kumar[*]
Microsoft Copilot AI
Redmond, WA
anjikumar@microsoft.com

Abstract

The LLM assistants a developer uses at the terminal are generic and amnesiac: they run in a separate window, see only what is pasted into them, forget the project-specific command you corrected yesterday, and — on a remote GPU box — require an API key that leaks into that box's environment and shell history. We present **MARS**, a single-binary terminal editor, multiplexer, and built-in agent that closes this gap with *local memory*: because the agent is embedded where the user works, it retrieves two kinds of context a cloud model cannot reach — the user's own past commands, and the tool's own documentation — and injects them into the prompt. The retrieval is deliberately trivial: lexical BM25 over a local file, no embeddings, nothing that leaves the machine. In a constrained domain this simple mechanism has an outsized effect. On personalized commands, feeding a small, inexpensive model (claude-haiku-4-5) a few *paraphrases* of the user's own past corrections raises translation accuracy from **37% to 95%** under a leakage-controlled protocol — the store holds only reworded prior requests, so a win requires retrieval to bridge a paraphrase gap, not look up a planted answer. Pointed at the tool's own documentation, the same mechanism lifts self-knowledge accuracy from **42% to 75%** (and mapping a plain-English request to the right configuration knob from **16% to 93%**). We describe the agent's minimal machinery (a registry-validated directive protocol, a corrective-memory loop, a free-model cascade), the architecture that makes always-there local memory possible (a persistent daemon and a key-never-leaves-home SSH broker), and a self-instrumenting evaluation in which the agent's debug log is the benchmark corpus.

[*] Work done before joining Microsoft.

1 Introduction

Consider an ML researcher training models across a laptop and a handful of remote GPU boxes. Over a day they babysit long runs in a shell, edit configs and model code in an editor, hold several sessions open with a multiplexer, and ask an LLM to explain a CUDA out-of-memory trace or draft an `rsync` command. The assistant is the weakest link, for two reasons that compound. It is **generic**: living in a separate window, it sees only the fragment the user pastes in, and does not know that in *this* project a training run is launched with a particular `sbatch` incantation settled on last week. And it is **amnesiac**: it forgets that command between sessions, forgets "where was I" on reattaching to an overnight run, and knows nothing about the editor it is embedded in. Worse, in the remote setting that defines GPU work, reaching the work at all means exporting an API key onto a box the researcher does not own, where it lingers in the environment, shell history, and often a dotfile.

Our insight is that an agent *embedded at the terminal* has access to memory a cloud model structurally cannot reach. Two local corpora sit within arm's length of a terminal-resident agent and nowhere near a browser tab: the user's own history of commands they actually ran, and the tool's own documentation, action registry, and configuration schema. Neither is large, secret to the machine, or requires training — they only require an agent that lives where they live. We built **MARS** (a Memory-Augmented Rust Shell) to exploit exactly this: it retrieves from both corpora with plain lexical search and injects the results into the prompt.

The surprise is how little machinery this takes and how much it buys. We expected local memory to help; we did not expect a trivial lexical method to nearly close the gap between a small, inexpensive model and its personalized ceiling — and to

do so even when the stored memory is only a *paraphrase* of the user's earlier request rather than its verbatim text. On a set of project-specific commands, retrieving a handful of reworded past corrections lifts claude-haiku-4-5 from 37% to 95% accuracy, with the retriever forced to bridge the paraphrase gap rather than echo a planted answer. In a narrow domain, memory need not be sophisticated to be decisive.

This paper makes four contributions. First, a memory-augmented terminal agent (§2) built from minimal parts — a registry-validated directive protocol, a corrective-memory loop, and a free-model cascade. Second, two axes of *local* memory: the user's own commands for translation, and the tool's own ground truth for self-knowledge and reconfiguration. Third, the architecture (§3) that makes always-there local memory possible: a persistent daemon and a key-never-leaves-home SSH broker that keeps the credential off remote boxes. Fourth, a self-instrumenting evaluation (§4) in which the agent's debug log doubles as the benchmark corpus, and in which corrective memory takes a small model from 37% to 95% on personalized commands under a leakage-controlled protocol. MARS is open source (MIT), installs with `cargo install mars-terminal`, and is about 9,500 lines of Rust.

2 The Memory-Augmented Agent

2.1 Tasteful minimalism

The recurring risk in an agentic editor is that the agent becomes a second system bolted beside the first, with its own command language, state, and failure modes. Our design principle is the opposite: add the *minimum* machinery that makes memory augmentation work, and reuse everything else. The agent is not a subsystem but another way of naming an action. Everything MARS can do is a variant of one `Action` enum, and three interfaces resolve a user's intent to it — a direct keychord (procedural recall), a fuzzy command bar (recognition), and the agent (natural language) — all terminating in a single dispatch point. A capability added once is therefore chord-bindable, searchable, and agent-invokable for free (Figure 1 sketches the retrieval loop).

This trades expressive freedom for grounding. Because the agent can only name an action that exists in the registry, and retrieves from flat local files rather than a vector index, the surface a user must trust is small and legible. We pay for this in ceiling: MARS cannot invent a capability at runtime, and lexical retrieval will miss a paraphrase an embedding would catch (§4). In the terminal domain the trade is strongly favorable, and the rest of this section inventories the few parts we did add.

Directive	Effect (gated by confirm)
`RUN: <Action>`	fire a registry action
`TYPE: <cmd>`	type a command into a shell pane
`OPEN: path:line`	jump to a file location
`NEED: <context>`	pull more context; re-ask once

Table 1: The directive protocol. A `RUN:` naming an action absent from the live registry is rejected before it fires, so a hallucinated action cannot execute; a destructive action passes the same confirmation gate a human would.

2.2 The agent as a validated input device

An agent that can act on the user's workspace must be grounded so it cannot act on a capability that does not exist. The principle, following the mode-error tradition in interface design, is that the agent is one input device among several and must be *safer* than a keypress, never a bypass around the editor's gates. The implementation is a trailing-line directive protocol: the model receives a live-generated catalog of every action, its description, and its keybinding, and ends its reply with at most one directive (Table 1). We chose a text protocol over provider-native tool-calling APIs deliberately — it works identically across OpenAI-compatible endpoints, a native Anthropic path, and local open models, and renders as human-readable text the user confirms before anything fires — at the cost of the stricter typing a schema gives.

The grounding is the registry check. A `RUN:` whose name does not resolve against the live `Action` set is rejected before dispatch, so the model cannot execute a hallucinated capability — the same enum that defines what MARS can do defines what the agent may invoke. `NEED:` closes a different gap: rather than always dumping the full screen, the model can request more context (a pane's scrollback, another tab) on demand, and MARS supplies it and re-asks exactly once, the depth hard-capped so no retrieval loop forms. This is a small, model-driven form of the retrieval this paper is about, applied to the live screen rather than to memory.

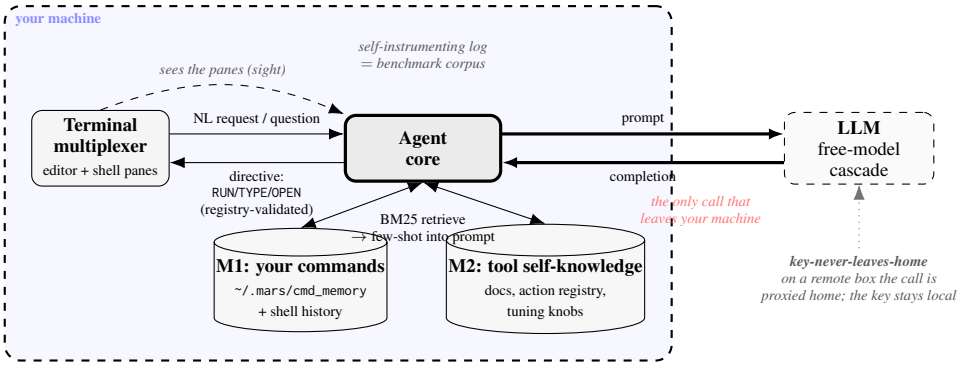

Figure 1: MARS augments a terminal LLM agent with *local memory over the user's own context*. Seeing the multiplexer's panes, the agent retrieves with lexical BM25 from two local stores — **M1**, the user's past commands, and **M2**, the tool's own docs and registry — and injects the results as few-shot exemplars. Only the model call leaves the machine (on a remote box even the key is proxied home), and every call is logged into the corpus the evaluation is built from. The dashed boundary is the thesis: memory, keys, and benchmark all stay local.

2.3 Natural-language shell translation

The most frequent agentic request at a terminal is "turn this English into the right command." The principle is to keep the human in the loop cheaply: translation should be assistive and reversible, never an automatic execution. The implementation lives on the command bar — `Ctrl+Space`, then plain English ("find the biggest checkpoints here") — which sends the request with the on-screen context (working directory, recent output) to the model cascade (§2.6) and renders the proposed command at the cursor to run or discard. Translation alone is commodity capability (Lin et al., 2018, 2017); what makes it work *for a given user* is the memory we turn to next.

2.4 Memory 1: the user's own commands

A generic model does not know your commands. Asked to "launch the training run," it produces a plausible `python train.py` when this project in fact uses `sbatch --gres=gpu:8 scripts/train.slurm` — a command the user has typed a dozen times and the model has never seen. The principle is that the user's own accepted commands are the highest-value context available, and are local to the terminal by construction. The implementation is a corrective loop. Every time the user accepts (or edits and runs) a translated command, MARS appends the `(request, command)` pair to a local JSON-lines file, and also reads recent shell history from `$HISTFILE`. On the next request, MARS ranks these entries against the query with BM25 (Robertson and Zaragoza, 2009) — k_1=1.5, b=0.75, no index to build — and injects the top few as few-shot exemplars.

The loop is *corrective* in the precise sense that a command the model got wrong once becomes one it gets right thereafter, because the user's correction is what enters the store — a capability a stateless cloud assistant cannot have, since it never observed the correction and does not persist between calls. Nothing here is novel as information retrieval; the contribution is that in this domain the retrieval can be this trivial and still be decisive (§4). The cost is a bounded prompt inflation — a few exemplars added to the input, which §4 measures and finds far outweighed by the accuracy gain (and on a reasoning model the grounding can even shorten the output enough to pay it back).

2.5 Memory 2: the tool's own documentation

An agent embedded in a tool should answer questions about that tool, and change its settings, from ground truth rather than a guess. The principle is that MARS's own capability surface — the action registry, keybindings, and tuning knobs, each carrying a human-written description — is authoritative documentation generated from the code, and cannot drift. The implementation reuses the same BM25 retriever over a second corpus: MARS's documentation chunked into passages, plus the live registry and the described knobs. On a `NEED: docs` pull, MARS injects the top passages, so "how do I split the screen?" is answered with the real keybinding and "make the accent orange" resolves to the actual knob and a confirmable edit. When retrieval returns nothing, the agent says the capability does not exist rather than invent one — honest refusal grounded in the same registry that gates `RUN:`.

2.6 Why simple memory is enough

It is worth stating why a method this trivial suffices, because the instinct in a memory system is to reach for embeddings and a vector database. The terminal is a *constrained* domain. A user's command vocabulary is small, repetitive, and lexically stable — the same flags, paths, and tool names recur verbatim — so query and useful memory share surface tokens, exactly the regime in which BM25 is strong and dense retrieval's paraphrase advantage is marginal. The corpora are small (hundreds of entries), so no scaling pressure justifies an index. And the payoff is asymmetric: a single relevant exemplar of the user's own command is worth more than semantic breadth, because it supplies the exact string the user wants reproduced. We therefore treat embeddings as a cost to be deferred until the domain broadens (§6). The empirical case for this choice is the 37%-to-95% paraphrase-memory result of §4 — where BM25 bridges a mean word-Jaccard gap of 0.36 between the query and the stored exemplar — and the mechanism producing it is a prompt, a local file, and a standard ranking function.

2.7 The model cascade

Most terminal LLM traffic is cheap and can run on free models, but a fixed choice of one hits rate limits and wastes quality where it is not needed. The principle is to right-size per task, distinguishing two orthogonal reasons to change models. Rotating *for limits* moves a task horizontally across same-tier open models on a rate-limit error: because each family is metered on its own counter, rotating across N families multiplies the effective free ceiling by roughly N, never lowering quality because it stays in-tier. Escalating *for quality* moves a task vertically to a stronger tier, only on a *detected* failure — which MARS gets for free, because its output is structured: a `RUN:` that fails the registry check is an unambiguous signal a small model erred, triggering one retry at a higher tier. This trades a little dispatch complexity for the free-tier headroom the token wall of §4.4 makes concrete.

3 System: The Substrate for Local Memory

Local memory is only useful if the agent is reliably present where the memory lives, across disconnects and on machines the user does not own. The architecture exists to guarantee that presence, and is kept as small as the agent.

One binary, one core. MARS is a single Rust binary. Its application core owns all state — buffers, panes, terminals, the agent conversation, the memory stores — and never touches a TTY directly: input arrives as source-neutral events, and rendering is a stateless projection onto a `ratatui` backend. The same core therefore runs unmodified in three configurations — a standalone terminal, a headless daemon, and a test backend — which lets the evaluation drive the real system with no mocks (§4).

A persistent daemon. Persistence is a process split, not a save-and-restore feature. A server process runs the whole application headless and emits the editor's own ANSI byte stream over a Unix socket; a thin client owns the real terminal and pumps bytes. We chose to have the server render ANSI rather than define a structured protocol because it reuses the entire existing rendering pipeline with no second renderer to maintain; the cost is that the wire carries rendered frames rather than semantic events. The payoff for memory is direct: because the core keeps running with no client attached, the command store and the user's session survive a closed laptop, and a run left going overnight is watched and summarized for the user's return.

Classifier-free routing. The command bar routes by *sigil*, not by a learned classifier: the first character names the target namespace — `!` runs a shell command, `?` asks the agent, `@` opens the file navigator, and plain text fuzzy-searches actions. Routing is deterministic and never mispredicts, so there is no routing model to train, evaluate, or explain, and no classification error for the user to work around.

The key never leaves home. The remote-work deficit is a security problem: exporting a key onto every GPU box scatters the credential across machines the user does not control. Our principle is that a secret is a fact about the user, not the machine, so it should live in one place. The implementation is a broker. A home-side daemon (`mars keyd`) holds the key; when the user runs `mars ssh`, MARS reverse-forwards the broker's socket over the SSH connection, and the agent on the remote box makes its model calls by proxying

the request *home*, where the broker injects the credential and streams the completion back. The key never lands on the remote — not on disk, in the environment, or in memory — so a compromised box has nothing to steal, and access ends when the tunnel closes.

The log is the benchmark. Running with `-llm-debug` appends one JSON line per model call — task, model, real token counts, latency, and the full prompt and reply — to a local stream, and records whether each suggestion was accepted, edited, or rejected. This log is both a cost profile the user can inspect and, as §4 shows, the corpus the evaluation is built from: the system instruments itself, so the benchmark is drawn from real use rather than authored by hand.

4 Evaluation

4.1 Setup

We evaluate the two memory axes on frozen, a-priori gold sets and a single small model under test. The command-translation gold set contains 112 items — 88 general and 24 project-specific (personalized) — and the self-knowledge gold set contains 82 question–answer items (52 knowledge, 30 reconfiguration); both were fixed before any system was run against them. The model under test is claude-haiku-4-5, a small, inexpensive model: MARS's thesis needs the model to be *small* — so personalization has a gap to close — not necessarily free (§4.4 explains why the final run uses a small paid model). To avoid a model grading itself, correctness is scored by a stronger cross-model judge, claude-sonnet-5 (Zheng et al., 2023). Prompts, contexts, and token counts come from the self-instrumenting log of §3; across all runs there were 0 empty completions.

4.2 Axis A: learning the user's commands

The base numbers in Table 2 confirm the premise of the paper. The small model is already good at general shell translation — 89% — but falls to 41% on project-specific commands it cannot know, dragging overall accuracy to 79%. These are not reasoning failures but missing-knowledge failures, exactly what local memory is positioned to fix.

The corrective-memory experiment isolates that fix, under a protocol designed to rule out the objection that memory is mere lookup. For each of the 24 tasks the store is seeded not with the prior request verbatim but with an independently generated *paraphrase* of it — a reworded request with the same intent (mean word-level Jaccard 0.36 to the test query) — while the model is tested on the *original* request. A "warm" win therefore requires retrieval to bridge the paraphrase gap and the model to generalize from a differently-worded exemplar, not to copy a planted answer. Under this setup, cold accuracy is 37% (9/24); paraphrase memory raises it to **95%** (23/24), with 15 of 24 items flipping from wrong-or-partial to correct; and a verbatim-memory condition, where the exact prior request is in the store, reaches the trivial lookup ceiling of 100% (24/24). The headline is the paraphrase number: simple BM25 over a local file recovers almost all of the personalization gap by generalizing, not by lookup. Memory adds a small token cost here (about 66/call), whose sign is model-dependent (§4.4, App. A).

Setting	Accuracy
General (n=88)	89%
Personalized (n=24)	41%
Overall (n=112)	79%
Corrective memory (personalized, n=24, leakage-controlled)	
Cold (no memory)	37%
Paraphrase memory	**95%**
Verbatim memory (ceiling)	100%

Table 2: Command translation (claude-haiku-4-5, judged by claude-sonnet-5). The corrective-memory block is leakage-controlled: the store holds only a *paraphrase* of each prior request and the model is tested on the original, so paraphrase memory (headline, 95%) measures generalization; verbatim memory (100%) is the trivial lookup ceiling.

4.3 Axis B: learning the tool itself

The second axis asks whether doc memory lets the agent answer questions about MARS and reconfigure it. Injecting retrieved documentation lifts self-knowledge accuracy from 42% to 75% overall (Table 3), but the gain is unevenly distributed. On "how-do-I" knowledge questions it is modest — 57% to 65% — because the no-memory model already answers many correctly (by naming the right action) and, even with docs, sometimes hallucinates a wrong keybinding for one the docs state exactly (§A). The decisive gain is on *reconfiguration* — "autosave more often" to a concrete knob edit — rising from 16% to **93%**: without the docs the model invents config filenames and knob names, while with the tuning-knob descriptions retrieved

Question type	No memory	+ Docs memory
Knowledge (how-do-I)	57%	65%
Reconfigure	16%	**93%**
Overall (n=82)	42%	**75%**

Table 3: Self-knowledge and self-reconfiguration (claude-haiku-4-5, judged by claude-sonnet-5). Docs memory helps knowledge questions modestly but is decisive on reconfiguration, where the retrieved knob descriptions supply the exact file, name, and default the model otherwise invents.

it emits the exact `knob = value in tuning.json` the user needs.

4.4 The free-tier token wall

The cascade's rationale surfaced as a hard constraint during evaluation itself. Our first choice for the model under test was a genuinely free one — but both free tiers we tried, Qwen3-32B on Groq and Gemini Flash-Lite, exhausted their *daily* token quotas partway through the roughly 350-call batch, the exact token wall the cascade (§2.7) routes around by rotating across metered families. Because a reproducible measurement needs every item scored in one clean pass, we ran the final numbers on a small *paid* model, claude-haiku-4-5 — small enough to stand in for the tier MARS targets (the thesis needs *small*, not free). Tellingly, the free-tier runs emitted empty completions on hitting the wall, whereas the paid run had 0 empties — confirming those blanks were a throughput artifact, not a comprehension failure. (A model-dependent effect of memory on token cost is in Appendix A.)

4.5 Limitations

Three limits bound these results honestly. The gold sets, though frozen and a-priori, are modest (112 and 82 items), so the personalized results read as "memory closes this gap on this set," not a claim at scale. Only one model is under test, so the numbers characterize the small, inexpensive regime MARS targets rather than models broadly; whether paraphrase generalization holds for a larger or reasoning model is open. And though we judge with a stronger model than the one under test (claude-sonnet-5 grading claude-haiku-4-5), an LLM judge still carries noise — a few points of grading slack either way (§A).

5 Related Work

Terminal tooling. Multiplexers such as tmux and Zellij give persistence but no intelligence; AI terminals such as Warp (Warp, 2024) add command search but see only their own blocks, not the user's editor or corrections; IDE agents such as Cursor (Anysphere, 2024) run their intelligence on the laptop while remote state lives on the box, and lose context when the window closes. MARS differs on one axis: the agent is embedded in a persistent, terminal-resident process, so it holds local memory none of these architectures host.

Natural language to commands. NL2Bash (Lin et al., 2018) and Tellina (Lin et al., 2017) established the NL-to-shell task and its corpora; we build on the task rather than the models, personalizing translation with the user's own history rather than improving the translator in general.

Agents and retrieval. Directive-style acting relates to ReAct (Yao et al., 2023) and tool use to Toolformer (Schick et al., 2023), though we deliberately use a portable text protocol with registry grounding rather than provider-native tool calls. Our memory is retrieval-augmented generation (Lewis et al., 2020) in its simplest lexical form; the novelty is not the mechanism but the setting — retrieval over the user's *own* local context, embedded where that context is produced.

6 Conclusion

A terminal agent embedded where the user works can retrieve two kinds of local memory — the user's own commands and the tool's own documentation — and in this constrained domain BM25 over a local file is decisive: a few reworded exemplars of the user's corrections take a small model from 37% to 95% on personalized commands, and the tool's own docs take it from 16% to 93% at reconfiguring itself. We read this as the first rung of a ladder. Above single-command recall lie *skill learning* (mining repeated command *sequences* into named procedures), *workflow understanding* (telling a training run from a debug session), and *cross-fleet memory* (embeddings, once paraphrase matters, following the user across machines through the same broker that forwards their key). The through-line is ownership: memory no cloud agent, seeing only what it is handed, can replicate. MARS is open source (Kumar, 2026).

References

Anysphere. 2024. Cursor: The AI code editor. https://cursor.com. Accessed July 2026.

Anjishnu Kumar. 2026. MARS: A memory-augmented agentic terminal multiplexer. https://crates.io/crates/mars-terminal. Software, installable via `cargo install mars-terminal`. Accessed July 2026.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Xi Victoria Lin, Chenglong Wang, Deric Pang, Kevin Vu, Luke Zettlemoyer, and Michael D. Ernst. 2017. Program synthesis from natural language using recurrent neural networks. *University of Washington Department of Computer Science and Engineering, Tech. Rep. UW-CSE-17-03-01*.

Xi Victoria Lin, Chenglong Wang, Luke Zettlemoyer, and Michael D. Ernst. 2018. NL2Bash: A corpus and semantic parser for natural language interface to the linux operating system. In *Proceedings of the Eleventh International Conference on Language Resources and Evaluation (LREC)*.

Stephen Robertson and Hugo Zaragoza. 2009. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389.

Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Warp. 2024. Warp: The intelligent terminal. https://www.warp.dev. Accessed July 2026.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. 2023. Judging LLM-as-a-judge with MT-bench and chatbot arena. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*.

A Qualitative Examples and Error Analysis

Every example in this appendix is quoted verbatim from the frozen result files the evaluation was scored on — `axis_a.jsonl` (112 translation items), `lift.jsonl` (24 leakage-controlled cold/paraphrase/verbatim items), and `axis_b.jsonl` (self-knowledge, 164 rows over the two memory conditions) — with the model under test claude-haiku-4-5 and the judge claude-sonnet-5 throughout. We organize the analysis by experiment, and within each by impressive behavior, unexpected errors, phenomena worth naming, and concrete recommendations. The point is not to celebrate the good cases but to show, from the raw data, exactly where a trivial local-memory mechanism helps, where it does not, and where the measurement itself is the limiting factor.

A.1 Axis A: NL→shell translation

Impressive behavior. On general commands the small model is a competent shell author, and its correct outputs are not only the easy one-liners: it composes multi-stage pipelines with the right utility at each stage, including summation and column projection in `awk`, buffered streaming, and process substitution. These were all judged correct:

```
a20  find the ten largest files under the cwd
   -> find . -type f -printf '%s %p\n' \
        | sort -rn | head -10
a66  count total lines across all python files
  -> find . -name "*.py" -type f -exec wc -l {} + \
        | tail -1
a07  kill the process listening on port 8080
   -> lsof -ti:8080 | xargs kill -9
a30  count how many times each unique line appears
   -> sort | uniq -c
a80  print columns 1 and 3 of a comma-separated file
   -> awk -F',' '{print $1,$3}' filename
```

This is the paper's premise made concrete: general shell translation is a near-solved commodity even on a small, inexpensive model (79 of 88 general items correct), so the accuracy that matters is the personalized slice the model cannot know from the request text alone.

Unexpected errors. Two error classes account for the general-command failures, and — unlike the free-tier runs of §4.4 — **this run produced zero empty completions**, so every failure is a real answer rather than a token-wall blank. First, **placeholder and precision misses**: the model guesses a stand-in filename the gold set spells differently (`a69` "print the contents of a file with line num-

bers" → `cat -n`, gold `cat -n file.txt`; `a31` `grep error log file` vs. `grep error app.log`), or picks a plausible but semantically different tool (`a38` "show my current IP" → `ip addr show`, a LAN address, where the reference wants `curl ifconfig.me`; `a86` reads from `/dev/stdin` where the gold splits a file into words). Second, on the personalized slice the dominant error is **ecosystem blindness** — the model answers a Rust or `uv` project with the statistically commonest ecosystem: `p04` "run the unit tests" → `npm test` (gold `cargo test`), `p06` "lint the code" → `eslint .` (gold `ruff check .`), `p11` "build the release binary" → `make release` (gold `cargo build --release`), `p18` "install project dependencies" → `npm install` (gold `uv sync`). A milder personalized pattern is **honest abstention**: on the hardest project commands the model declines rather than bluff (`p03` "deploy the model," `p12` "evaluate the latest checkpoint" → "I need more context…"), which is safe but still scored wrong against a gold command. Both are unknowable from the request text and are exactly what corrective memory targets.

Phenomena. Even under the stronger claude-sonnet-5 oracle, the judge assigns a *partial* band that captures near-misses, and reading it is informative: the general failures split into 6 outright wrong and 3 partial, and the partials are precisely the placeholder-filename cases (`a22` `chmod +x script` vs. gold `script.sh`; `a77` `zip -r archive.zip .` vs. `. . . folder/`) where the model understood the task but guessed a different stand-in. The cross-run grading instability that plagued earlier drafts is largely gone — the stronger judge grades the same cold output consistently across `axis_a.jsonl` and `lift.jsonl` — so the practical consequence is narrower than before: the Axis-A point estimates should be read with a couple of points of partial-credit ambiguity, not a large judge-noise band.

Recommendations. Two cheap fixes would tighten the measurement without changing the system's thesis. (1) Placeholder-normalized scoring, or explicit partial credit, so that a filename stand-in is not counted as a comprehension error — the 6 cold *partials* of the lift set (below) show how much signal lives in this band. (2) Marking honest abstentions distinctly from wrong guesses, since "I need more context" on a personalized command is a different failure mode from confidently emitting the wrong ecosystem. Neither touches the model or the memory mechanism — they are evaluation hygiene, and the earlier draft's retry-on-empty recommendation is now moot given the reliable model produced no blanks.

A.2 Corrective memory

Impressive behavior. This is the paper's headline result, and the per-item data shows the mechanism cleanly — crucially, under the leakage-controlled protocol where the store holds only a *paraphrase* of each prior request (mean word-Jaccard 0.36 to the test query). With those reworded exemplars retrieved by BM25 into the prompt, 23 of 24 project-specific items are correct, up from 9 correct cold; 15 items flip from wrong-or-partial to correct. Table 4 quotes representative flips verbatim. Two sub-mechanisms are visible. The first is **ecosystem correction**: a Rust or `uv`-native project answered with the commonest default (`npm`, `eslint`, `shellcheck`) is corrected to the user's real toolchain. The second is **exact-argument recall**: the model already had the right utility but the wrong specifics, and memory supplies the precise config, path, or split the user expects reproduced (`p02` "run the training job" gains `--config configs/base.yaml`; `p14` "run inference on the test set" becomes `python infer.py --split test`).

Unexpected errors. The paraphrase column has a single error out of 24, and it is the honest edge of the method: `p19` "run the benchmark suite" is answered `cargo bench` where the user's real command is `python -m benchmarks.run` — the model resolved the ecosystem to Rust and the retriever did not surface the paraphrased Python exemplar strongly enough to override it. This is a retrieval-plus-prior miss, not a lookup failure, and it is exactly the residual one expects when memory must generalize rather than copy. The *shape* of the cold column is also informative: of the 15 non-correct items, 9 are outright wrong and 6 partial, so memory is repairing a mix of fundamentally-wrong ecosystem guesses and last-mile precision misses, not merely polishing near-correct commands.

Phenomena. The most telling datum is the gap between the two warm conditions. Verbatim memory (the exact prior request in the store) scores 100% (24/24) — the trivial lookup ceiling — while paraphrase memory scores 95% (23/24).

Request	Cold (no memory)	+ Paraphrase memory
run the unit tests	`npm test`	`cargo test`
lint the code	`eslint .`	`ruff check .`
build the release binary	`make release`	`cargo build --release`
install project dependencies	`npm install`	`uv sync`
check code formatting	`shellcheck *.sh`	`black --check .`
clean the build artifacts	`rm -rf build/ dist/ . . .`	`cargo clean`
run the training job	`python train.py`	`python train.py --config configs/base.yaml`
deploy the model	*(asks for more context)*	`./scripts/deploy.sh --prod`

Table 4: Representative wrong/partial→correct flips from the leakage-controlled experiment (`lift.jsonl`). The store held only a *paraphrase* of each prior request (mean word-Jaccard 0.36) and the model was tested on the original, so a warm win requires generalization across the paraphrase gap, not lookup. The top block is *ecosystem correction* (a Rust/uv project answered with `npm`/`eslint`/`shellcheck` defaults); the last two rows are *exact-argument recall* and *abstention rescue*. All warm outputs match gold.

That the paraphrase condition falls only one item short of verbatim lookup, despite a mean word-Jaccard of just 0.36 between the stored exemplar and the query, is the clean measure of what BM25 buys here: the retriever bridges a substantial rewording gap on 23 of 24 items, so the personalization win is a genuine generalization result rather than an artifact of a planted answer. The one-item gap is the honest price of that generalization.

Recommendations. The residual is a *retrieval* miss, not a generation miss: when the paraphrase drifts far enough and a strong ecosystem prior competes, lexical BM25 does not surface the stored pair decisively. The cheapest next lever, given that stored commands now carry `ts`/`session`/`cwd`, is recency-and-project weighting — when several exemplars tie lexically, prefer the user's *recent* and *same-directory* commands. Semantic retrieval (embeddings) is the rung after that, warranted only once the domain broadens enough that the paraphrase gap dominates, per §6.

A note on token cost. Memory's effect on token cost is *model-dependent*, an insight that cuts against a claim an earlier draft of this work made. On non-reasoning claude-haiku-4-5, retrieved exemplars add a small input cost (mean 95→161 tokens/call, +66). But on a *reasoning* model (Qwen3-32B, from the free-tier runs of §4.4) the same few-shot grounding suppresses the verbose chain-of-thought the model otherwise emits when unsure, so net cost *falls* (385→217 per call, −44%). Memory as a token *reduction* is thus a reasoning-model phenomenon, not a universal property; we report the honest per-model effect rather than headline the favorable case.

A.3 Axis B: self-knowledge

Docs memory lifts overall accuracy from 42% to 75% (35→62 of 82), but the two question types behave very differently, and the split is the story. Retrieval turns a plausible guess into a grounded fact:

```
k02 "How do I save the current file?"
 none: "Use the Save action."
      [would run: Save]            (acts, no key)
 docs: "Press C-x C-s to save
        the current buffer."          (correct)
r16 "How do I change the accent color?"
 none: "Mars doesn't have built-in
        theme color settings ..."     (wrong)
 docs: set theme_accent to an [r,g,b]
       array in tuning.json           (correct)
```

The reconfigure slice is where memory is decisive: 16% to **93%** (5→28 of 30). Without docs the model invents plausible config locations — `r16` "change the accent color" is answered "Mars doesn't have built-in theme color settings," `r04` "use a different model" points at a `/.mars/config.json` that does not exist — whereas the retrieved knob descriptions carry the exact name, file, and default, so the agent answers `theme_accent` / `MARS_LLM_MODEL` in `tuning.json` correctly. Two failure families that motivated the corpus fix — **acts-instead-of-explains** (answering a how-to with a bare `[would run: . . . ]` directive, `k02`, `k09`) and **hallucinated identifiers** (a right-idea knob under the wrong filename) — are largely eliminated on this slice by the actionable, path-carrying knob descriptions and an explain-don't-act instruction on the docs path.

Knowledge questions gain far less — 57% to 65% — for an instructive reason: they turn on an *exact keybinding*, and even with docs retrieved the model sometimes hallucinates a plausible chord it

prefers over the one the docs state (k05 “detach my session” → C-x C-c, gold C-x C-d; k06 “search within a buffer” → C-f, gold C-s; k18 “jump to top/bottom” → C-x Home, gold M-<). A knob *name* is a token the retriever can hand over intact; a chord is a specific fact the model will still overwrite with a more common default, which is why doc memory helps reconfiguration far more than keybinding recall. A residual limit is judge slack: a few correct-but-differently-phrased answers are marked wrong, so the reported knowledge number is a mild lower bound even under the stronger claude-sonnet-5 oracle.